Use BFS to solve a planning problem. Examples can be 8 puzzle problem, Robot path planning,Blocks world etc.

```
from collections import deque



def bfs(grid, start, goal):

    rows = len(grid)
    cols = len(grid[0])

    queue = deque([start])
    visited = {start}
    parent = {start: None}

    # Up, Down, Left, Right
    directions = [
        (-1, 0),
        (1, 0),
        (0, -1),
        (0, 1)
    ]

    while queue:

        current = queue.popleft()

        if current == goal:
            break

        for dr, dc in directions:
```

```
        nr = current[0] + dr
        nc = current[1] + dc

        next_position = (nr, nc)

        if (
            0 <= nr < rows and
            0 <= nc < cols and
            grid[nr][nc] != 1 and
            next_position not in visited
        ):

            visited.add(next_position)
            parent[next_position] = current
            queue.append(next_position)

# Check if goal was reached
if goal not in parent:
    return None

# Create shortest path
path = []
current = goal

while current is not None:
    path.append(current)
    current = parent[current]

path.reverse()
```

```
    return path



# 0 = Free path
# 1 = Obstacle

grid = [
    [0, 0, 0, 1, 0],
    [1, 0, 0, 1, 0],
    [0, 0, 1, 0, 0],
    [0, 0, 0, 0, 0]
]

# Starting and Goal positions
start = (0, 0)
goal = (3, 4)

# Find shortest path
path = bfs(grid, start, goal)

if path:
    print("Shortest Path:")
    print(path)
    print("Number of moves:", len(path) - 1)

else:
    print("No path found")

#RUN
python bfs_maze.py
```